

Занятие 6. Диффузия для текста: сначала discrete bridge, потом LLaDA

Какой вопрос закрывает занятие

В занятиях про Flow Matching и diffusion мы всё время работали с непрерывным состоянием. Объект x_τ жил в евклидовом пространстве, можно было прибавить Gaussian noise, определить плотность $p_\tau(x)$, взять её gradient и получить score $s_\tau(x) = \nabla_x \log p_\tau(x)$.

Для текста такой перенос не проходит автоматически. Токен - это categorical variable. Если слово cat имеет id 100, а dog id 101, из этого не следует, что они находятся на расстоянии 1 в каком-либо содержательном пространстве. Можно перенумеровать vocabulary, язык от этого не изменится, а численные расстояния между id изменятся полностью.

Поэтому главный вопрос занятия такой:

Что остаётся от diffusion и Flow Matching, когда состояние дискретно и обычного $\nabla_x \log p(x)$ больше нет?

Ответ состоит из нескольких частей. Вместо Gaussian corruption можно построить discrete corruption, например постепенно заменять токены на специальный absorbing state **MASK**. Вместо continuous score модель может предсказывать distribution исходного токена. Вместо vector field в continuous Flow Matching можно рассматривать transition rates, которые перемещают probability mass между дискретными состояниями. После этого masked diffusion становится полноценной generative construction, а не просто «BERT, который много раз запускают».

На этом мосте можно уже честно обсуждать большие diffusion language models, в частности LLaDA. При этом важно не потерять системную сторону. Bidirectional denoising даёт возможность предсказывать несколько позиций параллельно, но не превращает generation в один forward pass. У обычной full-sequence dLLM каждый denoising step снова обрабатывает последовательность целиком. Поэтому тезис «у diffusion почти нет memory cost, потому что нет KV cache» неверен: отсутствие cache означает в том числе отсутствие reuse вычислений. Block diffusion как раз возвращает cache, меняя структуру зависимостей.

Что нужно знать заранее

Нужны конструкции из занятий 4 и 5:

- probability path p_τ ;
- направление diffusion time τ : $\tau = 0$ соответствует clean data, $\tau = 1$ - noise, a generation идёт в обратную сторону;
- Gaussian corruption и marginal score;
- denoising targets;
- vector field и continuity equation;
- различие numerical solver step и model evaluation;
- N_{FE} как число model/function evaluations;
- понимание Transformer attention и KV cache.

Из занятия 1 пригодится ещё одна договорённость: cost нельзя сворачивать в одно слово «память». Persistent state, workspace, HBM traffic, FLOPs, количество обработанных token positions и последовательная глубина generation - разные величины.

Что должно получаться после чтения

После этого конспекта должно получаться:

1. Объяснить через permutation vocabulary ids, почему token id не задаёт естественную евклидову геометрию.
2. Объяснить, почему one-hot representation сама по себе не спасает обычный continuous score.
3. Записать masked corruption kernel и вывести distribution числа замаскированных позиций.
4. По clean sequence самостоятельно построить один training example для masked diffusion.
5. Объяснить, что именно предсказывает probability denoiser.
6. Назвать и объяснить четыре содержательных отличия masked diffusion от классического BERT recipe.
7. Переписать Rao-Blackwellized MDLM weight из keep-probability notation в notation этого курса.
8. Объяснить master equation как conservation law для probability mass на discrete state space.
9. Объяснить, чем transition rate отличается от transition probability.
10. Объяснить, почему LLaDA показывает возможность масштабировать masked diffusion до large LM, но сама по себе не доказывает превосходство diffusion над autoregression.
11. Объяснить, почему bidirectional generation допускает parallel token prediction, но всё ещё требует iterative denoising.
12. Объяснить, почему standard exact KV cache естественен для causal Transformer и ломается при full bidirectional denoising.
13. Вывести block-autoregressive factorization и показать, откуда в block diffusion снова появляется cache.
14. Не смешивать sampler steps, N_{FE} , число обработанных token positions за один NFE, block size и число blocks.

15. Правильно пересказать few-step result: «в такой-то конфигурации авторы уменьшили число sampling steps» вместо «diffusion LM генерирует ответ за четыре вызова сети».

Словарь занятия

Понятие	English term	Определение
Дискретное пространство состояний	discrete state space	Пространство, элементы которого являются отдельными категориями или комбинациями категорий, а не точками непрерывного евклидова пространства.
Категориальная переменная	categorical variable	Random variable, принимающая одно из конечного набора значений $\mathcal{V}$.
Перенумерация словаря	vocabulary relabeling	Замена token ids произвольной permutation без изменения семантики данных.
One-hot представление	one-hot representation	Представление категории v вершиной e_v стандартного simplex в $\mathbb{R}^V$.
Дискретное зашумление	discrete corruption	Stochastic process, переводящий clean categorical state в повреждённое дискретное состояние.
Masked diffusion	masked diffusion	Discrete diffusion, где noise реализован заменой clean categories на специальный mask state.
Поглощающее состояние	absorbing state	State, из которого forward corruption больше не выходит. Для masked diffusion таким state является MASK .
Вероятность маскирования	corruption probability / masking probability	ρ_τ - вероятность, что clean token уже заменён на MASK к noise level τ .
Уровень шума	noise level	Параметр τ , определяющий степень corruption.
Обратное восстановление	reverse denoising	Движение от heavily corrupted state к clean data.
Probability denoiser	probability denoiser	Модель, предсказывающая categorical distribution clean state по corrupted state.
Discrete probability path	discrete probability path	Семейство distributions p_τ на discrete state space.
Скорость перехода	transition rate	Instantaneous rate $r_\tau(x \rightarrow y)$ перехода между discrete states в CTMC.

Понятие	English term	Определение
Непрерывная по времени марковская цепь	continuous-time Markov chain, CTMC	Markov process на discrete state space с continuous time и переходами, задаваемыми rates.
Master equation	master equation	Equation evolution probability mass через incoming и outgoing transition flows.
Diffusion language model	dLLM	Language model, где generation задаётся iterative denoising process, а не только left-to-right factorization.
Двунаправленный denoising	bidirectional denoising	Denoising, где prediction позиции может использовать видимый контекст и слева, и справа.
Iterative unmasking	iterative unmasking	Постепенное превращение masked positions в token values за несколько generation steps.
Parallel token prediction	parallel token prediction	Получение predictions для нескольких positions из одного model evaluation.
Block diffusion	block diffusion	Factorization, autoregressive между blocks и diffusion-based внутри каждого block.
KV cache	KV cache	Persistent storage key/value representations, позволяющее не пересчитывать неизменившийся context.
Шаг sampler	sampler step	Один алгоритмический шаг reverse generation procedure.
NFE	number of function evaluations	Число фактических evaluations neural network или другой моделируемой функции.
Block size	block size	Число token positions внутри одного diffusion block.

1. Почему continuous score нельзя механически перенести на токены

Зачем эта тема здесь

До сих пор score был центральным математическим объектом diffusion: $s_\tau(x) = \nabla_x \log p_\tau(x)$.

Если сразу написать аналог для текста, легко получить формулу, которая выглядит знакомо, но не определена в том же смысле.

Сначала нужно понять, какая именно структура потерялась при переходе от изображения или continuous latent к token sequence.

Интуиция

Пусть vocabulary состоит из V токенов: $\mathcal{V} = \{v_1, \dots, v_V\}$.

Tokenizer присваивает им integer ids. Но id нужен программе для indexing. Это label категории, а не координата категории.

Например, допустим:

- cat имеет id 100;
- dog имеет id 101;
- banana имеет id 500.

Если смотреть только на числа, можно решить, что cat и dog «ближе», чем cat и banana.

Теперь переставим ids:

- cat получает id 100;
- banana получает id 101;
- dog получает id 500.

Текстовые distributions, значения предложений и задача language modeling от этого не меняются. Изменилось только внутреннее соглашение tokenizer.

Значит, геометрия вида $d(v_i, v_j) = |i - j|$ вообще не является свойством языка.

Ниже мы даже не будем использовать эту формулу как допустимую distance: она нужна только для демонстрации ошибки.

Понятия, которые вводятся

Здесь нужны:

- discrete state space;
- categorical variable;
- vocabulary relabeling;
- permutation invariance;
- one-hot representation;
- continuous relaxation.

Формальное изложение

1.1. Последовательность токенов живёт в конечном пространстве

Пусть $X = (X_1, \dots, X_T)$, $X_t \in \mathcal{V}$.

Тогда всё пространство последовательностей фиксированной длины равно $\mathcal{X} = \mathcal{V}^T$.

Это конечное множество.

В continuous diffusion ситуация была другой. Например, при $x \in \mathbb{R}^d$ две близкие точки x и $x + \delta$ имеют естественный геометрический смысл. Можно спрашивать, как меняется $\log p(x)$ при бесконечно малом сдвиге δ .

Для token id такого бесконечно малого движения нет. Между category cat и category dog не существует состояния cat + 0.01.

1.2. Перестановка ids уничтожает искусственную геометрию

Пусть $c: \mathcal{V} \rightarrow \{1, \dots, V\}$ задаёт integer id каждого token.

Попробуем определить Gaussian-like corruption прямо на ids:

$$q_{\sigma}(v' | v) \propto \exp \left(-\frac{(c(v') - c(v))^2}{2\sigma^2} \right).$$

Такая формула делает близкими tokens с близкими ids.

Теперь возьмём произвольную permutation $\pi : \{1, \dots, V\} \rightarrow \{1, \dots, V\}$ и перенумеруем vocabulary: $c_\pi(v) = \pi(c(v))$.

После этого тот же самый рецепт дал бы

$$q_{\sigma, \pi}(v' | v) \propto \exp \left(- \frac{(c_\pi(v') - c_\pi(v))^2}{2\sigma^2} \right).$$

Для общей permutation значения расстояний меняются, следовательно,

$$q_{\sigma, \pi}(v' | v) \neq q_\sigma(v' | v)$$

для части пар v, v' .

Но underlying language distribution не изменилась вообще. Мы только переставили labels.

Получается, такой noise process зависит от arbitrary implementation detail. Это хороший diagnostic test для constructions над categorical variables:

Если произвольная согласованная permutation category labels меняет содержательную модель, скорее всего в construction случайно появилась геометрия ids.

1.3. Почему нельзя просто написать $\nabla_x \log p(x)$

Пусть X принимает значения в конечном $\mathcal{X}$.

Probability mass function $p(x) = \Pr(X = x)$ определена только на точках $x \in \mathcal{X}$.

Обычный gradient требует функцию, определённую хотя бы в некоторой continuous neighbourhood точки. Чтобы взять $\nabla_x \log p(x)$, нужно сначала решить, что такое $p(z)$ между discrete states.

Но это уже дополнительный modeling choice. Одну и ту же probability mass function на vertices можно продолжить в ambient continuous space множеством способов, и у этих extensions будут разные gradients.

Поэтому обычный Euclidean score не является canonical object для token ids.

Это не утверждение, что в discrete generative modeling вообще нет объектов, называемых score. Например, можно строить probability ratios между states или другие discrete analogues. Утверждение уже: continuous score нельзя получить бесплатно, просто объявив ids координатами.

1.4. А если использовать one-hot?

Более разумная идея - представить token v one-hot vector $e_v \in \mathbb{R}^V$.

Здесь исчезает проблема близких integer ids. Любая permutation vocabulary просто переставляет coordinates, а разные one-hot vertices симметричны.

Но возникает другая проблема.

Categorical distribution теперь задаёт measure

$$\mu = \sum_{v \in \mathcal{V}} p(v) \delta_{e_v},$$

где δ_{e_v} - point mass в vertex e_v .

Эта distribution сконцентрирована на конечном наборе points. У неё нет обычной smooth density относительно V -мерного Lebesgue measure. Значит, объект $\nabla_z \log p(z)$ в ambient space всё ещё не определён так, как он был определён для Gaussian-corrupted continuous data.

1.5. Gaussian noise поверх one-hot уже определяет другую модель

Можно сделать следующий шаг:

$$p_\sigma(z) = \sum_{v \in \mathcal{V}} p(v) \mathcal{N}(z; e_v, \sigma^2 I).$$

Теперь $z \in \mathbb{R}^V$, density smooth, и $\nabla_z \log p_\sigma(z)$ существует.

Это математически нормальная construction. Но обратите внимание, что мы сделали: state generation process теперь continuous vector z , а не categorical token.

После denoising всё равно требуется определить, как из continuous z получить token. Можно взять nearest vertex, categorical decoder или ещё одну learned distribution. Всё это дополнительные choices.

Такой подход можно называть diffusion над continuous relaxation или embeddings. Нельзя только сказать: «мы записали tokens как one-hot, поэтому ordinary discrete diffusion автоматически получила continuous score».

1.6. Learned embeddings тоже задают выбранную геометрию

Ещё один вариант - взять embedding matrix и считать token представителем некоторой точки $e_v \in \mathbb{R}^d$.

После этого Gaussian corruption формально возможен.

Но расстояния между tokens теперь зависят от learned embedding geometry. Это может быть хорошим modeling choice, но это уже выбранная geometry, а не intrinsic geometry token ids.

В masked diffusion можно вообще не делать такой выбор. Corruption действует непосредственно на categories.

Пример

Пусть vocabulary состоит всего из трёх категорий: $\mathcal{V} = \{A, B, C\}$.

В первой нумерации: $c(A) = 1$, $c(B) = 2$, $c(C) = 100$.

Gaussian-like kernel на ids считает A и B очень близкими.

После permutation:

$$c_{\pi}(A) = 1, \quad c_{\pi}(C) = 2, \quad c_{\pi}(B) = 100.$$

Теперь тот же kernel считает близкими A и C .

Data distribution над symbols A, B, C при этом не изменилась. Изменилось только encoding.

Именно поэтому integer difference token ids не может играть роль $x - y$ из обычной Gaussian diffusion.

Где здесь легко ошибиться

Ошибка 1. «Token ids - числа, значит можно добавить Gaussian noise».

Числовой datatype не делает label координатой.

Ошибка 2. «One-hot лежит в $\mathbb{R}^V$, значит score уже определён».

Нет. Discrete distribution на vertices не становится smooth density только из-за того, что vertices записаны vectors.

Ошибка 3. «Если Gaussian smoothing one-hot работает, значит это и есть discrete diffusion».

Нет. После Gaussian smoothing state continuous. Это допустимая отдельная architecture, но нужно отдельно определять связь с categorical output.

Ошибка 4. «У tokens совсем нет геометрии».

Можно специально построить geometry через embeddings, graph structure, semantic metric и так далее. Вопрос не в запрете, а в том, что geometry не содержится в ids автоматически.

Что теперь понятно

Непрерывная diffusion construction опиралась не только на слово «noise». Она опиралась на differential structure пространства.

У categorical tokens этой структуры нет. Значит, нужно выбрать generative mechanism, естественный именно для discrete states.

Самый простой вариант для language modeling - не двигать token по искусственной числовой оси, а иногда переводить его в специальное состояние «значение скрыто».

2. Masked / absorbing-state corruption

Зачем эта тема здесь

Нам нужен discrete аналог forward corruption, который:

1. определён непосредственно на tokens;
2. не зависит от arbitrary token ids;
3. имеет понятные endpoints;
4. позволяет построить reverse denoising task.

Masked diffusion удовлетворяет всем четырём условиям.

Интуиция

Вместо

слегка сдвинуть значение token в random continuous direction

мы делаем

с некоторой вероятностью забыть, какой token здесь стоял.

Добавим category **MASK**.

При небольшом noise level скрыта малая доля sequence. При большом - почти вся. В крайней точке вся sequence состоит из masks.

Forward process стирает информацию: $x \longrightarrow \tilde{x}_\tau$.

Generation идёт обратно:

$$\tilde{x}_1 \longrightarrow \tilde{x}_{\tau_1} \longrightarrow \dots \longrightarrow x.$$

Модель в этом случае не обязана предсказывать continuous noise vector. Естественная задача - восстановить distribution скрытого clean token по оставшемуся context.

Понятия, которые вводятся

- masked diffusion;
- absorbing state;
- masking probability ρ_τ ;
- noise schedule;
- reverse denoising;
- masked-set random variable.

Формальное изложение

2.1. Расширяем vocabulary

Пусть clean vocabulary равно $\mathcal{V}$.

Для corruption введём

$$\bar{\mathcal{V}} = \mathcal{V} \cup \{\text{MASK}\}.$$

Clean data всё ещё лежат в $\mathcal{V}^T$. Corrupted states лежат уже в $\bar{\mathcal{V}}^T$.

Для одной position зададим marginal corruption kernel

$$q_\tau(\tilde{x}_{\tau,t} \mid x_t) = (1 - \rho_\tau)\mathbf{1}[\tilde{x}_{\tau,t} = x_t] + \rho_\tau\mathbf{1}[\tilde{x}_{\tau,t} = \text{MASK}].$$

Здесь $\rho_0 = 0$, $\rho_1 = 1$.

Поэтому при $\tau = 0$ $\tilde{X}_0 = X$, а при $\tau = 1$

$$\tilde{X}_1 = (\text{MASK}, \dots, \text{MASK})$$

с вероятностью 1.

Это сохраняет convention занятия 5: increasing τ означает data-to-noise. Generation проходит path в обратном направлении.

2.2. Независимое маскирование всей sequence

Самая простая forward construction маскирует positions независимо conditional on X :

$$q_\tau(\tilde{X}_\tau | X) = \prod_{t=1}^T q_\tau(\tilde{x}_{\tau,t} | X_t).$$

Введём indicator

$$I_{\tau,t} = \mathbf{1}[\tilde{X}_{\tau,t} = \text{MASK}].$$

Тогда

$$I_{\tau,t} \sim \text{Bernoulli}(\rho_\tau),$$

и при условии independent corruption эти variables независимы по t .

Число masked positions:

$$N_{\text{mask},\tau} = \sum_{t=1}^T I_{\tau,t}.$$

2.3. Вывод distribution числа masks

Зафиксируем m .

Сначала выберем некоторый конкретный набор S из m positions. Вероятность, что именно эти m positions masked, а остальные $T - m$ остались clean, равна $\rho_\tau^m (1 - \rho_\tau)^{T-m}$.

Сколько существует subsets размера m ?

$$\binom{T}{m}.$$

Эти events взаимно исключают друг друга, поэтому probabilities складываются:

$$\Pr(N_{\text{mask},\tau} = m | X) = \binom{T}{m} \rho_\tau^m (1 - \rho_\tau)^{T-m}.$$

Следовательно,

$$N_{\text{mask},\tau} \sim \text{Binomial}(T, \rho_\tau).$$

Distribution не зависит от значений самих tokens. Только от T и noise level.

Для expectation:

$$\begin{aligned}\mathbb{E}[N_{\text{mask},\tau} \mid X] &= \mathbb{E}\left[\sum_{t=1}^T I_{\tau,t}\right] \\ &= \sum_{t=1}^T \mathbb{E}[I_{\tau,t}] = T\rho_\tau.\end{aligned}$$

Для variance используем independence:

$$\begin{aligned}\text{Var}(N_{\text{mask},\tau} \mid X) &= \sum_{t=1}^T \text{Var}(I_{\tau,t}) \\ &= T\rho_\tau(1 - \rho_\tau).\end{aligned}$$

Так что ρ_τ имеет очень прямую interpretation: expected fraction masked positions равна ρ_τ .

2.4. Marginal corruption ещё не полностью задаёт trajectory

Формула для $q_\tau(\tilde{X}_\tau \mid X)$ говорит, как выглядит state на каждом noise level отдельно.

Но из набора marginals ещё не следует, как состояния на разных τ связаны в одной trajectory.

Чтобы получить именно absorbing process, при monotone ρ_τ удобно построить coupling.

Для каждой position один раз сэмплируем $U_t \sim \text{Uniform}[0, 1]$.

Дальше говорим:

$$\tilde{X}_{\tau,t} = \text{MASK} \iff U_t \leq \rho_\tau.$$

Тогда

$$\Pr(\tilde{X}_{\tau,t} = \text{MASK}) = \rho_\tau,$$

то есть нужный marginal сохраняется.

При этом если ρ_τ не убывает, то после попадания в **MASK** position уже не может снова стать clean при дальнейшем увеличении τ . Поэтому **MASK** действительно является absorbing state forward process.

Это полезное различие:

- corruption marginal отвечает, насколько noisy state на конкретном τ ;
- coupling отвечает, как один noisy state превращается в другой по мере изменения τ .

2.5. Что делает reverse process

Возьмём два уровня $0 \leq \sigma < \tau \leq 1$.

Если position masked на уровне τ , то для reverse generation нас интересует вероятность, что на более clean уровне σ она уже должна быть открыта.

В threshold coupling мы знаем $U_t \leq \rho_\tau$.

Чтобы position быть clean на уровне σ , требуется $U_t > \rho_\sigma$.

Поэтому

$$\begin{aligned} & \Pr(U_t > \rho_\sigma \mid U_t \leq \rho_\tau) \\ &= \frac{\rho_\tau - \rho_\sigma}{\rho_\tau} = 1 - \frac{\rho_\sigma}{\rho_\tau}. \end{aligned}$$

Здесь предполагается $\rho_\sigma \leq \rho_\tau$.

Итак, reverse process должен решить две задачи:

1. какие masked positions сделать visible на следующем более clean level;
2. какие clean token values туда поставить.

Schedule и sampler отвечают за первую часть. Learned denoiser даёт distribution для второй.

2.6. Как получается один training example

Training example для masked diffusion строится так.

Берём clean sequence $X \sim p_{\text{data}}$.

Сэмплируем noise level $\tau \sim p_{\text{train}}(\tau)$.

Вычисляем masking probability ρ_τ .

Для каждой position независимо сэмплируем

$$I_{\tau,t} \sim \text{Bernoulli}(\rho_\tau).$$

Дальше

$$\tilde{X}_{\tau,t} = \begin{cases} \text{MASK}, & I_{\tau,t} = 1, \\ X_t, & I_{\tau,t} = 0. \end{cases}$$

В model input идёт $\tilde{X}_{\tau}$, а clean X_t остаётся training target на masked positions.

Важно, что мы не строим trajectory через сотни intermediate states для каждого training example. Как и в continuous diffusion, если marginal corruption известен в closed form, можно сразу sample state на случайном τ .

Пример

Пусть $T = 10$ и $\rho_{\tau} = 0.3$.

Тогда $N_{\text{mask},\tau} \sim \text{Binomial}(10, 0.3)$, поэтому $\mathbb{E}[N_{\text{mask},\tau}] = 3$ и

$$\text{Var}(N_{\text{mask},\tau}) = 10 \cdot 0.3 \cdot 0.7 = 2.1.$$

Вероятность получить ровно три masks:

$$\Pr(N_{\text{mask},\tau} = 3) = \binom{10}{3} 0.3^3 0.7^7 \approx 0.267.$$

Например,

the model predicts several tokens in parallel

может превратиться в

the MASK predicts MASK tokens in MASK .

Denoising model получает corrupted sequence целиком и должна восстановить distributions clean tokens в masked positions.

Где здесь легко ошибиться

Ошибка 1. «При $\rho_{\tau} = 0.3$ всегда masking ровно 30% positions».

Нет. При independent masking 30% - expectation. Фактическое $N_{\text{mask},\tau}$ random.

Ошибка 2. «Формула marginal corruption автоматически означает absorbing trajectory».

Нет. Для trajectory нужна joint construction across times. Threshold coupling выше - один конкретный способ получить absorbing process с нужными marginals.

Ошибка 3. «На training надо прогнать forward chain до случайного τ ».

Если marginal известен, state можно sample напрямую.

Ошибка 4. «Reverse generation просто снимает masks независимо».

Нет. Token values должны зависеть от context. Именно эту conditional distribution учит denoiser.

Что теперь понятно

Masked diffusion даёт настоящий discrete probability path без искусственной metric между tokens.

Следующий вопрос: loss очень похож на masked language modeling. Почему тогда это не просто BERT?

3. MDLM и чем это не BERT

Зачем эта тема здесь

На уровне одной corrupted sequence training действительно выглядит знакомо:

- часть tokens заменили на **MASK**;
- Transformer видит context;
- на masked positions считаем cross-entropy.

Если остановиться здесь, masked diffusion можно ошибочно принять за переименование BERT.

Но generative model задаётся не только local loss. Важны probability path, distribution noise levels, weighting objective и reverse sampler.

Интуиция

Можно использовать одну и ту же elementary prediction task внутри совершенно разных probabilistic constructions.

Например, задача

по контексту предсказать пропущенный token

сама по себе ещё не говорит:

- с какой вероятностью возникают разные mask rates;
- как weighted examples разных noise levels;
- определяет ли objective likelihood bound;
- существует ли reverse generative chain;
- от какого initial state начинается generation.

В MDLM local masked-token cross-entropy встроена в diffusion construction.

В этом и состоит основное отличие.

Понятия, которые вводятся

- probability denoiser;
- weighted masked-language-model loss;
- Rao-Blackwellized objective;
- SUBS parameterization;
- reverse sampler.

Формальное изложение

3.1. Общий weighted objective

Будем писать $p_\theta(x_t \mid \tilde{X}_\tau, \tau)$ для model distribution clean token в position t .

Общий вид objective:

$$\mathcal{L} = \mathbb{E}_{\tau, X, \tilde{X}_\tau} \left[w(\tau) \sum_{t: \tilde{x}_{\tau, t} = \text{MASK}} -\log p_\theta(x_t \mid \tilde{X}_\tau, \tau) \right].$$

Здесь есть три разных источника randomness:

1. clean sample X ;
2. noise level τ ;
3. mask pattern $\tilde{X}_\tau$ conditional on X, τ .

Weight $w(\tau)$ определяет относительный вклад different noise levels.

3.2. Что является optimal prediction target

Зафиксируем noise level τ , corrupted context $\tilde{X}_\tau = z$ и masked position t .

Обозначим true conditional distribution clean token:

$$\eta_v = \Pr(X_t = v \mid \tilde{X}_\tau = z, \tau), \quad v \in \mathcal{V}.$$

Пусть model выдаёт probabilities q_v .

Expected cross-entropy в этой фиксированной ситуации:

$$R(q) = - \sum_{v \in \mathcal{V}} \eta_v \log q_v.$$

При этом

$$q_v \geq 0, \quad \sum_v q_v = 1.$$

Используем KL decomposition, которая не требует предположения об interior optimum:

$$R(q) = - \sum_{v \in \mathcal{V}} \eta_v \log q_v = H(\eta) + D_{\text{KL}}(\eta \| q).$$

Здесь $H(\eta)$ не зависит от q , а стандартная convention $0 \log 0 = 0$ включает distributions на границе simplex.

Так как $D_{\text{KL}}(\eta \| q) \geq 0$, minimum достигается при $q = \eta$. Это утверждение включает границу simplex; если $\eta_v = 0$, соответствующая component не требует interior stationarity argument.

Получается, population optimum masked-token cross-entropy - это posterior distribution clean token:

$$p_\theta(x_t \mid \tilde{X}_\tau, \tau) \approx p_{\text{data}}(x_t \mid \tilde{X}_\tau, \tau).$$

Это и есть probability denoiser.

Обратите внимание на разницу с continuous score. Model output здесь не vector direction в token-id space, а categorical distribution.

3.3. Что делает weight

Если бы для каждого τ существовала отдельная infinitely expressive model, любой strictly positive scalar $w(\tau)$ не менял бы conditional optimum на этом τ .

Но реальная model одна и та же для всех noise levels, имеет finite capacity и обучается stochastic optimization. Поэтому weighting определяет, как сильно разные regions probability path конкурируют за capacity и gradient budget.

Это та же общая мысль, что в занятии 5: формально связанные denoising targets и practically equivalent training recipes - не одно и то же.

3.4. Exact MDLM weight в notation этого курса

В исходной MDLM notation keep probability часто обозначается α_τ . В notation этого курса для той же величины используем κ_τ , чтобы не конфликтовать с signal coefficient занятия 5.

В MDLM удобно сначала использовать keep probability $\kappa_\tau = 1 - \rho_\tau$.

То есть κ_τ - вероятность, что token всё ещё clean.

Для continuous-time Rao-Blackwellized MDLM objective paper получает weight перед masked-token negative log-likelihood

$$w_{\text{MDLM}}(\tau) = \frac{\dot{\kappa}_\tau}{\kappa_\tau - 1}.$$

Поскольку $\kappa_\tau = 1 - \rho_\tau$, имеем $\dot{\kappa}_\tau = -\dot{\rho}_\tau$ и $\kappa_\tau - 1 = -\rho_\tau$.

Поэтому

$$w_{\text{MDLM}}(\tau) = \frac{\dot{\rho}_\tau}{\rho_\tau}.$$

Это именно результат MDLM derivation; полный NELBO derivation здесь не воспроизводим. Paper выводит simplified Rao-Blackwellized objective как mixture masked-language-model losses.

Для linear schedule $\rho_\tau = \tau$ получается

$$w_{\text{MDLM}}(\tau) = \frac{1}{\tau}.$$

На $\tau = 0$ expression singular. Но сама вероятность увидеть masked position при linear schedule также порядка τ . Поэтому expected weighted contribution не обязан расходиться: вероятность появления соответствующего term компенсирует factor $1/\tau$. В continuous objective отдельная endpoint имеет measure zero; implementation обычно samples interior noise levels.

Формула выше предполагает запись integral по $d\tau$, либо эквивалентный uniform sampling τ . Если τ samples из другого proposal density, Monte Carlo weight должен также учитывать этот proposal.

3.5. Что означает Rao-Blackwellized

Rao-Blackwellization в общем виде означает: часть latent randomness аналитически marginalize вместо того, чтобы каждый раз sample её в estimator.

Conditional expectation сохраняет mean estimator и не увеличивает variance.

В MDLM этот приём вместе со специальной reverse parameterization приводит к более простой objective, которая выглядит как weighted family обычных MLM losses. Авторы отдельно называют две полезные properties parameterization: clean prediction не должна выдавать mask category, а уже unmasked token может быть carried over вместо повторного resampling.

3.6. Четыре отличия от BERT

Теперь можно строго ответить на главный вопрос раздела.

Отличие 1. Noise level является random variable generative path

В классическом BERT recipe выбирается фиксированная доля candidate positions. В original BERT это 15% positions, после чего для выбранных positions применялся дополнительный 80/10/10 recipe: mask token, random token или unchanged token.

Mask pattern при этом может пересэмплироваться. «Фиксированный» здесь означает фиксированный corruption regime, а не один и тот же mask set навсегда.

В masked diffusion samples $\tau \sim p(\tau)$, после чего mask rate равен ρ_τ .

Model поэтому видит целый continuum corruption levels: от почти clean до почти completely masked.

Это не cosmetic difference. Эти levels являются points одного generative probability path.

Отличие 2. Weighting связано с diffusion objective

BERT MLM objective обычно представляет собой cross-entropy на выбранных prediction positions.

В MDLM вклад разных noise levels появляется из diffusion likelihood-bound construction. Для рассмотренного continuous-time masked process получаем, например,

$$w_{\text{MDLM}}(\tau) = \frac{\dot{\rho}_\tau}{\rho_\tau}.$$

Значит, одинаковые по форме token-wise cross-entropies входят в другой global objective.

Отличие 3. Denoiser является частью reverse sampler

После BERT pretraining можно, конечно, придумать iterative fill-in algorithm. Но такой sampler не является автоматически probabilistic inverse того training corruption recipe.

В MDLM reverse generative mechanism является частью model definition. Probability denoiser используется, чтобы реально двигаться от noisy distribution к data distribution.

То есть вопрос

«Как из model получить sample?»

для diffusion не external add-on, а часть construction.

Отличие 4. Generation стартует с сильно испорченного state

BERT во время обычного использования получает в основном наблюдаемый текст и решает representation/prediction task.

Masked diffusion generation может начинаться с

$$\tilde{X}_1 = (\text{MASK}, \dots, \text{MASK})$$

и затем многократно уменьшать noise.

MDLM project page прямо иллюстрирует generation как начало с all-mask sequence и последовательное восстановление tokens.

3.7. Ещё одно, менее фундаментальное отличие

В basic absorbing-state diffusion forward corruption действительно переводит selected token в **MASK**.

Original BERT 80/10/10 recipe часть selected positions оставлял unchanged, а часть заменял random token.

Это конкретное отличие corruption kernels, но оно не заменяет четыре пункта выше. Даже если сделать BERT corruption чисто masked, generative construction всё равно не станет MDLM автоматически.

Пример

Пусть training sequence имеет длину 100.

В BERT-style setup можно каждый раз выбирать около 15 candidate positions.

В masked diffusion один example может получить $\rho_\tau = 0.04$, другой $\rho_\tau = 0.51$, третий $\rho_\tau = 0.93$.

Model учится решать один family denoising tasks на radically different information levels.

На $\rho_\tau = 0.04$ почти весь context известен.

На $\rho_\tau = 0.93$ большую часть sequence приходится восстанавливать, опираясь на очень sparse evidence и learned language prior.

Reverse sampler затем использует именно эту способность двигаться через разные noise regimes.

Где здесь легко ошибиться

Ошибка 1. «MDLM - это BERT с большим mask rate».

Нет. Mask rate только один из четырёх differences.

Ошибка 2. «Если local loss совпадает, probabilistic model совпадает».

Нет. Нужно учитывать distribution τ , weight, forward path и sampler.

Ошибка 3. « $1/\tau$ означает infinite training loss около нуля».

Не автоматически. При linear masking event «position masked» сама имеет probability τ .

Ошибка 4. «Rao-Blackwellization - это ещё одна architecture Transformer».

Нет. Это способ упростить stochastic objective за счёт analytical marginalization.

Ошибка 5. «BERT вообще нельзя использовать генеративно».

Можно строить generative procedures поверх masked LM. Здесь утверждение другое: standard BERT recipe сам по себе не задаёт MDLM reverse diffusion process.

Что теперь понятно

Discrete diffusion не нуждается в continuous score, чтобы иметь principled denoising objective.

Natural model target - posterior distribution clean category.

При этом идея probability path намного шире конкретного masked corruption. Следующий шаг - показать connection с Flow Matching без попытки восстановить Euclidean vector field.

4. Discrete Flow Matching

Зачем эта тема здесь

В занятии 4 Flow Matching строился вокруг трёх объектов:

1. probability path;
2. local dynamics, задающей движение samples;
3. conservation equation, гарантирующего нужную evolution distribution.

Может показаться, что вся эта схема требует vectors и ordinary velocities.

На самом деле нет. Что действительно важно - transport probability mass.

В continuous space mass течёт через infinitesimal spatial neighbourhoods. В discrete space она перескакивает между states.

Интуиция

Представим три states: A , B , C .

Вместо velocity vector в state A можно задать rates:

- из A в B ;
- из A в C ;
- из других states обратно в A .

Probability $p_\tau(A)$ растёт от incoming transitions и уменьшается от outgoing transitions.

То есть continuous continuity equation заменяется discrete accounting rule:

изменение probability в state = incoming mass rate - outgoing mass rate.

Это и есть master equation.

Понятия, которые вводятся

- discrete probability path;
- transition rate;
- continuous-time Markov chain;
- master equation;
- probability denoiser.

Формальное изложение

4.1. Probability path не обязан жить в $\mathbb{R}^d$

Пусть $\mathcal{S}$ - конечное discrete state space.

Это может быть:

- vocabulary одной position;
- множество mask/clean states;
- целое пространство token sequences;
- более структурированное discrete object space.

Probability path - это просто family

$$p_\tau(x), \quad x \in \mathcal{S}, \quad \tau \in [0, 1].$$

Для каждого τ

$$p_\tau(x) \geq 0, \quad \sum_{x \in \mathcal{S}} p_\tau(x) = 1.$$

Никакого derivative по coordinate x здесь не требуется.

4.2. Transition rate

Для разных states $x, y \in \mathcal{S}$ зададим $r_\tau(x \rightarrow y) \geq 0$.

Rate не является probability.

Его meaning задаётся small time increment h :

$$\Pr(X_{\tau+h} = y \mid X_\tau = x) = r_\tau(x \rightarrow y)h + o(h), \quad y \neq x.$$

Вероятность остаться в x :

$$\begin{aligned} & \Pr(X_{\tau+h} = x \mid X_\tau = x) \\ &= 1 - h \sum_{y \neq x} r_\tau(x \rightarrow y) + o(h). \end{aligned}$$

Почему здесь появляется sum?

Потому что за infinitesimal interval probability уйти из x - это sum probabilities переходов во все остальные states.

Rate сам по себе может быть больше 1. Требование probability применяется к product $r_\tau h$ при sufficiently small h .

4.3. Вывод master equation

Хотим найти $p_{\tau+h}(x)$.

На time $\tau + h$ оказаться в x можно двумя способами.

Первый - уже быть в x и не уйти:

$$p_\tau(x) \left(1 - h \sum_{y \neq x} r_\tau(x \rightarrow y) \right).$$

Второй - прийти из некоторого $y \neq x$:

$$\sum_{y \neq x} p_\tau(y) r_\tau(y \rightarrow x) h.$$

С точностью до terms $o(h)$:

$$\begin{aligned} p_{\tau+h}(x) &= p_\tau(x) \left(1 - h \sum_{y \neq x} r_\tau(x \rightarrow y) \right) \\ &+ h \sum_{y \neq x} p_\tau(y) r_\tau(y \rightarrow x) + o(h). \end{aligned}$$

Вычтем $p_\tau(x)$:

$$\begin{aligned} p_{\tau+h}(x) - p_\tau(x) &= -h p_\tau(x) \sum_{y \neq x} r_\tau(x \rightarrow y) \\ &+ h \sum_{y \neq x} p_\tau(y) r_\tau(y \rightarrow x) + o(h). \end{aligned}$$

Делим на h и берём limit $h \rightarrow 0$:

$$\partial_\tau p_\tau(x) = \sum_{y \neq x} [p_\tau(y)r_\tau(y \rightarrow x) - p_\tau(x)r_\tau(x \rightarrow y)].$$

Это master equation.

Первый term внутри brackets - incoming probability mass.

Второй - outgoing probability mass.

4.4. Rates самого masked corruption

Теперь свяжем general CTMC с masked path из предыдущего раздела. Рассмотрим одну position, которая при $\tau = 0$ находится в конкретном clean state x . Пусть ρ_τ не убывает и $\rho_\tau < 1$.

Forward-процесс имеет единственный ненулевой outgoing rate из clean state:

$$r_\tau^{\rightarrow}(x \rightarrow \mathbf{MASK}) = \frac{\dot{\rho}_\tau}{1 - \rho_\tau}.$$

Все остальные outgoing rates из clean x равны нулю, а **MASK** является absorbing state.

Для clean state master equation даёт

$$\partial_\tau p_\tau(x) = -p_\tau(x) \frac{\dot{\rho}_\tau}{1 - \rho_\tau}.$$

При initial condition $p_0(x) = 1$ решение равно $p_\tau(x) = 1 - \rho_\tau$. Для mask state incoming flow даёт

$$\partial_\tau p_\tau(\mathbf{MASK}) = \dot{\rho}_\tau,$$

поэтому $p_\tau(\mathbf{MASK}) = \rho_\tau$. Так конкретный masked corruption является частным CTMC, а не отдельной от master equation construction.

4.4. Почему total probability сохраняется

Просуммируем master equation по всем x :

$$\begin{aligned} \partial_\tau \sum_x p_\tau(x) &= \sum_x \sum_{y \neq x} p_\tau(y)r_\tau(y \rightarrow x) \\ &\quad - \sum_x \sum_{y \neq x} p_\tau(x)r_\tau(x \rightarrow y). \end{aligned}$$

В первом double sum просто переименуем indices x и y .

Тогда он становится ровно тем же набором ordered transitions, что и второй double sum.

Следовательно, они взаимно уничтожаются:

$$\partial_\tau \sum_x p_\tau(x) = 0.$$

Если в начале probabilities sum to one, это остаётся верно для всего path.

Это discrete conservation law.

4.5. Аналогия с continuous Flow Matching

В continuous space continuity equation имела структуру

$$\partial_\tau p_\tau + \nabla \cdot (p_\tau v_\tau) = 0.$$

Здесь v_τ говорит, куда локально течёт mass.

В discrete space вместо spatial divergence получаем sum incoming/outgoing flows: $p_\tau(x)r_\tau(x \rightarrow y)$.

То есть conceptual correspondence такое:

Continuous	Discrete
point $x \in \mathbb{R}^d$	category/state $x \in \mathcal{S}$
velocity $v_\tau(x)$	transition rates $r_\tau(x \rightarrow y)$
local spatial flux	probability mass transferred between states
continuity equation	master equation
vector-field learning	learning a denoiser/rates that induce the desired path

Это и есть настоящий bridge к занятию 4. Мы сохранили probability path и conservation principle, но поменяли local mathematical object.

Discrete Flow Matching в частности строит general probability paths для discrete data и связывает sampling с learned posteriors, в том числе probability denoiser, то есть x -prediction.

4.6. Probability denoiser вместо score

Один естественный learned object:

$$D_\theta(v \mid z, \tau) \approx \Pr(X_0 = v \mid X_\tau = z).$$

Для sequence это может быть token-wise categorical distribution, conditional on whole corrupted sequence.

Discrete Flow Matching показывает, как из таких learned posteriors строить rates для выбранного probability path. Полный sampler здесь не выводим: для этого пришлось бы разбирать конкретную parameterization path и scheduler.

Главная идея уже видна: model не обязана выдавать «direction между token ids». Она может выдавать probabilities clean categories, а transport law переводит эти probabilities в discrete transitions.

4.7. Не перепутать направление τ

В этом конспекте сохраняется convention занятия 5:

- $\tau = 0$ - clean data;
- $\tau = 1$ - maximally corrupted state;
- generation идёт от 1 к 0.

В Flow Matching literature нередко path параметризуют наоборот: source at 0, target data at 1.

Master equation от этого не меняет meaning. Меняется только orientation path и соответствующий знак/направление dynamics.

Поэтому connection с занятием 4 - structural:

probability path + local transport law + conservation,

а не обязательное совпадение смысла endpoints $\tau = 0$ и $\tau = 1$.

Пример

Пусть $\mathcal{S} = \{A, B, \text{MASK}\}$.

Для reverse generation введём отдельный clock $s = 1 - \tau$, который растёт от noisy state к clean data, и обозначим $p_s^{\text{rev}}(z) = p_{1-s}(z)$. Предположим, при этом reverse process может перейти из **MASK** либо в **A**, либо в **B**.

Тогда rate изменения probability state **A** содержит, среди прочего,

$$p_s^{\text{rev}}(\text{MASK})r_s^{\text{rev}}(\text{MASK} \rightarrow A).$$

Это incoming flow.

Одновременно, если reverse dynamics допускает такой transition, $p_s^{\text{rev}}(A)r_s^{\text{rev}}(A \rightarrow \text{MASK})$ является outgoing flow.

Никакой numeric distance между A и B не понадобилось.

Нужно только определить allowed transitions и их rates.

Где здесь легко ошибиться

Ошибка 1. «Discrete Flow Matching - это continuous Flow Matching по token ids».

Нет. Transport object другой.

Ошибка 2. «Rate - это probability перейти за один step».

Не само по себе. Probability за short interval h имеет leading term $r_\tau h$.

Ошибка 3. «Master equation создаёт или уничтожает probability mass».

Нет. Incoming для одного state является outgoing для другого, поэтому total mass сохраняется.

Ошибка 4. «Probability denoiser и score - одно и то же число в другой notation».

Нет. Это разные mathematical objects. Между parameterizations иногда можно вывести transformations, но нельзя отождествлять их по определению.

Что теперь понятно

Основная идея diffusion/flow survives переход к discrete data.

He survives именно continuous geometry.

Вместо gradient field можно работать с categorical posteriors и transition flows. На этой основе уже имеет смысл обсуждать большие masked diffusion LMs.

5. LLaDA: масштабирование masked diffusion

Зачем эта тема здесь

До этого раздела discussion была в основном conceptual: мы построили корректный discrete generative bridge.

Теперь нужен concrete large-scale example.

LLaDA важна здесь не как доказательство, что diffusion «победила» autoregressive LM. Более аккуратный вывод такой:

masked diffusion удалось масштабировать до 8B language model, обученной с нуля в обычной pretraining + SFT paradigm.

Авторы LLaDA описывают 8B model, trained from scratch, с masking forward process и reverse masked-token prediction.

Это уже достаточно сильный факт для архитектурного курса: construction из предыдущих разделов не ограничивается toy models.

Интуиция

Autoregressive generation строит answer слева направо: $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots$.

Masked diffusion может начать с:

MASK MASK MASK MASK MASK

и затем получить, например:

MASK models MASK tokens MASK

потом:

Diffusion models MASK tokens parallel

и только после следующих iterations прийти к fully decoded sequence.

Ключевая возможность здесь - за один forward Transformer может вернуть distributions сразу для многих masked positions.

Но эти predictions сделаны относительно одного и того же current context. После того как часть predictions принята, context меняется. Чтобы остальные positions смогли воспользоваться новой information, требуется ещё один model evaluation.

Вот откуда одновременно возникают и parallelism, и iteration.

Понятия, которые вводятся

- diffusion language model, dLLM;
- bidirectional denoising;
- iterative unmasking;
- parallel token prediction;
- confidence-based token selection.

Формальное изложение

5.1. Training остаётся masked denoising

В simplified linear schedule можно взять

$$\rho_\tau = \tau, \quad \tau \sim \text{Uniform}(0, 1).$$

Тогда shape masked-diffusion objective содержит знакомый factor

$$\frac{1}{\tau}.$$

Схематически:

$$\mathcal{L} = \mathbb{E} \left[\frac{1}{\tau} \sum_{t: \tilde{X}_{\tau,t} = \text{MASK}} -\log p_\theta(X_t | \tilde{X}_\tau) \right].$$

Это та же family tasks:

- sample mask ratio;
- mask tokens;
- bidirectional Transformer видит corrupted sequence;
- predict original categories on masked positions.

Отдельный scalar τ не обязан всегда подаваться в network input буквально: corrupted pattern сам содержит information о noise level. Это parameterization detail, а не изменение probability path.

5.2. Generation начинается с masks

Для output region длины T initial state может быть $X^{(0)} = (\text{MASK}, \dots, \text{MASK})$.

На iteration m обозначим множество ещё masked positions через $\mathcal{M}_m$.

Один model evaluation выдаёт distributions

$$p_{\theta}(x_t \mid X^{(m)}), \quad t \in \mathcal{M}_m.$$

Sampler может получить candidate для каждой position:

$$\hat{x}_t^{(m)} \sim p_{\theta}(\cdot \mid X^{(m)}).$$

Дальше sampler не обязан сразу принять все candidates. Можно выбрать subset $A_m \subseteq \mathcal{M}_m$ по confidence или prescribed schedule и только для этих positions снять mask.

Тогда $\mathcal{M}_{m+1} = \mathcal{M}_m \setminus A_m$.

Следующий forward уже sees newly revealed tokens.

5.3. Почему можно предсказывать параллельно

Transformer с bidirectional attention может вычислить logits сразу для всех masked positions.

То есть один NFE может дать candidate tokens для многих positions.

Это принципиально отличается от strict left-to-right AR dependency, где token $t + 1$ обычно не генерируется до того, как принято значение token t .

5.4. Почему это не означает «один проход»

Рассмотрим две masked positions i и j .

В current state обе содержат **MASK**.

Model выдаёт $p_{\theta}(x_i \mid X^{(m)})$ и $p_{\theta}(x_j \mid X^{(m)})$.

Обе distributions conditioned на один и тот же state, в котором истинное новое value второй position ещё неизвестно.

Если мы одновременно sample $\hat{x}_i$ и $\hat{x}_j$, prediction i не могла использовать фактический newly sampled $\hat{x}_j$, а prediction j - newly sampled $\hat{x}_i$.

После того как часть tokens accepted, появляется new context: $X^{(m+1)} \neq X^{(m)}$.

Только новый forward позволяет remaining positions учитывать эту information.

Именно поэтому

bidirectional prediction $\neq$ one-pass joint generation.

Bidirectionality говорит, какие positions model может использовать как context в одном state. Iterative denoising говорит, сколько раз state обновляется.

5.5. Чем LLaDA важна и чем не важна

Paper представляет LLaDA 8B как masked diffusion LM, pre-trained from scratch, и показывает масштабирование standard pretraining/SFT recipe на diffusion objective.

Из этого корректно следует:

- large masked diffusion LM можно реально обучить;
- bidirectional denoising работает не только в small models;
- masked diffusion может демонстрировать nontrivial language capabilities.

Из этого не следует:

- diffusion universally better autoregression;
- один diffusion step equivalent одному AR token step;
- одинаковое parameter count означает одинаковый training compute;
- benchmark comparison автоматически изолирует effect generative factorization;
- bidirectional model генерирует arbitrary answer за один forward.

Если paper сравнивает со своими AR baselines, это empirical comparison в конкретных training conditions. Из неё нельзя получить theorem о model classes.

Пример

Пусть output состоит из восьми positions: $X^{(0)} = (M, M, M, M, M, M, M, M)$, где $M = \text{MASK}$.

После первого NFE sampler принимает positions 2 и 6: $X^{(1)} = (M, x_2, M, M, M, x_6, M, M)$.

После второго NFE model уже может использовать x_2 и x_6 при prediction всех остальных masks.

Допустим, принимаются positions 1, 5 и 8: $X^{(2)} = (x_1, x_2, M, M, x_5, x_6, M, x_8)$.

И так далее.

Первый NFE действительно дал predictions для восьми positions одновременно.

Но final sequence появилась не после одного NFE, потому что later predictions conditioned на increasingly informative context.

Где здесь легко ошибиться

Ошибка 1. «Non-autoregressive означает one-shot».

Нет. Non-autoregressive factorization и number of iterative refinement steps - разные axes.

Ошибка 2. «Все tokens predicted parallel, значит dependencies между ними потеряны полностью».

Внутри одного update candidates используют shared bidirectional context. Между iterations принятые tokens становятся context для остальных.

Ошибка 3. «8B LLaDA доказала superior diffusion scaling law».

Нет. Это demonstration feasibility and scale, не controlled theorem о превосходстве architecture class.

Ошибка 4. «Если несколько token predictions получены за один NFE, стоимость NFE та же, что у одного-token AR decode».

Нет. Full-sequence bidirectional Transformer за этот NFE обрабатывает намного больше token positions.

Что теперь понятно

Parallelism dLLM находится внутри одного refinement step.

Чтобы обсуждать speed, недостаточно считать, сколько tokens появилось после одного pass. Нужно считать как минимум:

- сколько model evaluations было;
- сколько token positions обработано в каждом;
- какую attention computation сделал каждый evaluation;
- что можно было reuse через cache.

И здесь появляется важная проблема full bidirectional dLLM.

6. Block diffusion, caching и малошаговость

Зачем эта тема здесь

Иногда преимущество diffusion формулируют так:

«KV cache не нужен, поэтому diffusion почти не требует memory».

Для language diffusion это плохая формулировка.

Standard full-sequence dLLM действительно обычно не имеет conventional exact AR KV cache. Но причина не в том, что прошлые computation magically не нужны. Наоборот: при bidirectional denoising они становятся invalid после изменения sequence, поэтому следующий step должен многое пересчитать.

Отсутствие reusable cache может уменьшить одну component persistent memory, одновременно увеличив repeated compute и memory traffic.

Block diffusion меняет dependency structure так, чтобы уже завершённый context снова стал cacheable.

Здесь две оси курса снова встречаются.

Ось А, контекст и backbone: что хранится, как читается, как записывается, что забывается.

Ось В, генеративная конструкция: как факторизуется распределение, какой путь, какой objective, какой сэмплер.

Block factorization меняет Ось В: generation становится autoregressive между blocks и diffusion-based внутри block. Именно это создаёт immutable completed prefix и тем самым меняет Ось А: для предыдущих blocks снова появляется exact reusable cache.

Интуиция

Почему KV cache работает в autoregressive Transformer?

Когда causal model уже обработала prefix

the diffusion model

и теперь генерирует следующий token, hidden state первого слова the не должен измениться из-за future token: causal mask запрещает ему смотреть в будущее.

Поэтому old keys и values prefix остаются valid.

В bidirectional dLLM ситуация другая.

Если state был

the MASK model

а потом стал

the diffusion model ,

token the имеет право attend к middle position. Значит, после изменения middle position representation the тоже может измениться.

И representation model тоже может измениться.

Получается, изменение одного token потенциально делает stale cache всех positions.

Block diffusion решает проблему structural way: прошлые blocks объявляются causal past, а bidirectional interaction оставляется внутри current block.

Понятия, которые вводятся

- full-sequence recomputation;
- block-autoregressive factorization;
- block diffusion;
- block-causal attention;
- hierarchical caching;
- few-step distillation;
- sampler steps;
- N_{FE} ;
- processed tokens per NFE;
- block size;
- number of blocks.

Формальное изложение

6.1. Почему exact AR KV cache работает

Рассмотрим causal Transformer.

Representation position i на layer ℓ зависит только от prefix: $h_i^{(\ell)} = f_i^{(\ell)}(x_1, \dots, x_i)$.

Если мы append future token x_{i+1} , causal attention запрещает position i смотреть на него.

Значит, $h_i^{(\ell)}$ для old prefix не меняется.

Следовательно, его $K_i^{(\ell)}$, $V_i^{(\ell)}$ тоже не меняются.

Их можно хранить и reuse на следующем decode step.

Это не просто engineering trick. Cache correctness следует из causal dependency graph.

6.2. Почему тот же argument ломается в bidirectional dLLM

Теперь attention внутри denoised region bidirectional.

Тогда уже на первом layer $h_i^{(1)}$ может зависеть от token x_j для любого visible j в sequence.

Если после одного denoising step $x_j^{(m+1)} \neq x_j^{(m)}$, то query position i на следующем pass видит другое value/key в j .

Поэтому generally $h_i^{(1,m+1)} \neq h_i^{(1,m)}$.

На следующем layer changed $h_i^{(1)}$ даёт changed $K_i^{(2)}$, $V_i^{(2)}$.

Так изменение одной position propagates через sequence.

Именно поэтому old full-sequence K/V tensors обычно нельзя считать exact cache следующего denoising state.

Block Diffusion paper прямо называет отсутствие KV caching одной из efficiency limitations bidirectional discrete diffusion и строит block formulation, которая её устраняет для previous blocks.

6.3. Почему из этого не следует «diffusion почти не требует memory»

Вернёмся к cost contract занятия 1:

$$\mathcal{C} = (M_{\text{weights}}, M_{\text{persistent}}, M_{\text{workspace}}, B_{\text{HBM}}, F, D_{\text{seq}}, N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}).$$

Отсутствие large persistent KV cache действительно может уменьшить $M_{\text{persistent}}$.

Но один full-sequence dLLM pass всё равно требует:

- model weights;
- current token/hidden representations;
- attention computation;
- intermediate workspace;
- reads/writes в HBM;
- processing sequence positions.

И таких passes несколько.

При vanilla full-sequence denoising каждый NFE снова обрабатывает sequence целиком с bidirectional attention. Если sequence length равна L , то rough token-processing count при N_{FE} evaluations:

$$N_{\text{processed}} \approx L N_{\text{FE}}.$$

Это не полный FLOP model - attention cost зависит от sequence length nonlinear way, а optimized kernels могут не materialize полный attention matrix. Но формула уже показывает, почему «нет KV cache» не означает «нет memory/computation problem».

Есть ещё одна тонкость. Denoising steps последовательны по времени, поэтому memory каждого step не суммируется в peak memory как $N_{\text{FE}} \cdot M_{\text{workspace}}$.

Workspace можно reuse.

То есть неверны обе крайности:

- «каждый step добавляет ещё один полный model state в peak memory»;
- «раз KV cache нет, memory почти не нужна».

Правильный ответ требует назвать конкретную component cost.

6.4. Block factorization

Разделим sequence на N_{blk} blocks: $X = (X^{(1)}, X^{(2)}, \dots, X^{(N_{\text{blk}})})$.

Из обычного chain rule для groups variables:

$$p(X) = p(X^{(1)})p(X^{(2)} | X^{(1)}) \dots p(X^{(B)} | X^{(1)}, \dots, X^{(B-1)}).$$

Обозначим previous blocks: $X^{(<b)} = (X^{(1)}, \dots, X^{(b-1)})$.

Тогда compactly:

$$p(X) = \prod_{b=1}^B p(X^{(b)} | X^{(<b)}).$$

Block diffusion использует autoregressive structure между blocks, но каждый conditional $p(X^{(b)} | X^{(<b)})$ реализует через discrete diffusion внутри block.

Именно так она интерполирует между двумя regimes:

- block size 1 приближает factorization к token-level autoregression;
- один block на всю output sequence даёт full-sequence diffusion regime.

Block Diffusion была опубликована на ICLR 2025; авторы подчёркивают flexible-length generation, parallel sampling внутри block и возможность KV caching.

6.5. Откуда возвращается cache

Attention pattern устроен асимметрично.

Current block имеет право:

- смотреть на completed previous blocks;
- bidirectionally взаимодействовать внутри себя.

Previous blocks при этом не зависят от current block.

Пусть completed prefix равен $X^{(<b)}$.

Пока мы многократно denoise current block $X^{(b)}$, prefix не меняется и его representations не зависят от changes current block.

Значит, для prefix keys и values можно вычислить один раз: $K_{\text{past}}, V_{\text{past}}$, после чего reuse их на denoising steps current block.

Inside current block exact static cache по той же причине не появляется автоматически: его tokens меняются и bidirectionally влияют друг на друга.

Когда block finally completed, он становится частью immutable past и его representations можно добавить в cache для future blocks.

Это отвечает на вопрос:

Зачем block diffusion возвращает cache, если diffusion якобы «cache не требует»?

Именно потому, что отсутствие cache было источником repeated computation. Block factorization intentionally создаёт invariant past, который можно не пересчитывать.

6.6. Почему block diffusion поддерживает arbitrary length

Whole-sequence diffusion часто предполагает заранее заданное число output positions, потому что initial state содержит соответствующее число masks.

Block-autoregressive model может генерировать blocks последовательно.

После completed block можно решить, продолжать ли generation, например через end-of-sequence mechanism.

Так что total output length не обязана быть известна целиком до первого denoising step.

Это ещё одно structural advantage block factorization, отдельно от caching.

6.7. Cache - не бесплатная победа

Возвращение KV cache увеличивает persistent state: $M_{\text{persistent}} \uparrow$.

Зато может уменьшить repeated computation и traffic:

$$F \downarrow, \quad N_{\text{processed}} \downarrow,$$

а иногда и effective $B_{\text{HBM}} \downarrow$ за счёт более выгодного reuse, хотя конкретный traffic зависит от implementation.

Поэтому «cache есть» и «cache нет» нельзя ранжировать без workload.

Cache - это tradeoff: хранить representations, чтобы не вычислять их снова.

6.8. Fast-dLLM v2 и hierarchical caching

Fast-dLLM v2 (arXiv 2025; ICLR 2026) развивает block-diffusion direction дальше.

Авторы используют block diffusion structure и hierarchical caching:

1. block-level cache для already completed history;
2. sub-block reuse внутри partially decoded block.

Вместе с parallel decoding pipeline это должно уменьшить redundant computation на inference. Авторы сообщают до **2.5×** speedup относительно standard autoregressive decoding в их experimental comparison, при сохранении generation quality по используемым ими metrics. Это именно measured result конкретной работы и setup, а не общий theorem «block diffusion всегда в 2.5 раза быстрее AR».

Здесь особенно полезно заметить изменение роли memory.

Vanilla full-sequence dLLM не сохраняет conventional exact KV state между denoising steps, но много пересчитывает.

Fast-dLLM-style system специально тратит persistent memory на caches, чтобы сократить repeated work.

Поэтому сама история развития dLLM уже противоречит slogan «diffusion cache просто не нужен».

6.9. Few-step generation и IDLM

Вторая ось ускорения - уменьшить число denoising steps.

IDLM, Inverse-distilled Diffusion Language Models, ICML 2026, distills pretrained many-step DLM в student, рассчитанный на fewer sampling steps. Авторы report reductions числа sampling steps на **4×-64×** в конкретных experiments; это claim о sampling depth соответствующих configurations, а не автоматически о N_{FE} или полной стоимости ответа.

В конкретных experiment configurations numbers разные.

Например, для IDLM-MDLM reported comparison сокращает sampling с 1024 до 16 steps. В отдельной IDLM-DCD + Greedy-Tail configuration в workshop results есть 4-step point, сравниваемый с 1024-step teacher.

Отсюда нельзя делать sentence:

«Diffusion language model генерирует ответ за четыре прохода».

Почему нельзя:

1. четыре - result конкретной distilled model и sampler configuration;
2. другая model в той же family использует 16, 256 или другое число steps;
3. sampling step не определён как один NFE;
4. даже если в конкретном sampler один step требует один model call, каждый call может обрабатывать много token positions;
5. benchmark result относится к определённой evaluation setting, а не к произвольному instruction-following answer.

Правильная формулировка выглядит примерно так:

В specific IDLM-DCD experiment с Greedy-Tail sampler авторы оценивают 4-step generation; для IDLM-MDLM в их reported low-step comparison используется 16 steps. Это сокращение sampling depth конкретных distilled models, а не универсальное свойство dLLM.

6.10. Пять величин, которые нельзя смешивать

Теперь разведём terminology строго.

Величина	Что считает
Sampler steps	Сколько algorithmic refinement transitions выполнил sampler.
N_{FE}	Сколько фактических network evaluations произошло.
Token positions per NFE	Сколько positions реально прошло через backbone в конкретном evaluation.
Block size L_{blk}	Сколько token positions относится к одному diffusion block.
Number of blocks N_{blk}	На сколько sequential blocks разбита generated sequence.

Sampler steps и NFE

Иногда один sampler step требует ровно один network evaluation: $N_{FE} = K$.

Но это property algorithm, не definition.

Step может включать:

- несколько model evaluations;
- predictor и corrector;
- conditional и unconditional passes;
- auxiliary scoring call;
- cache refresh.

Поэтому сначала смотрим actual algorithm, потом связываем steps с NFE.

NFE и processed tokens

Допустим, два systems имеют $N_{FE} = 16$.

Первая на каждом evaluation обрабатывает 1024 positions.

Вторая - только current block из 32 query positions плюс cached context.

Одинаковый NFE ещё не означает одинаковый compute.

Удобно определить

L_j = число **token positions**, заново обработанных на **NFE** j .

Тогда

$$N_{\text{processed}} = \sum_{j=1}^{N_{\text{FE}}} L_j.$$

Для vanilla full-sequence dLLM примерно $L_j \approx L$, поэтому

$$N_{\text{processed}} \approx L N_{\text{FE}}.$$

Для cached block system L_j может быть существенно меньше L .

Но и $N_{\text{processed}}$ не является полным compute metric: current queries всё ещё attend к cached keys, так что prefix length влияет на attention FLOPs и memory traffic.

Block size и number of blocks

Если output length L ровно делится на block size L_{blk} ,

$$B = \frac{L}{S}.$$

Это combinatorial structure sequence, а не number of denoising steps.

В каждом из N_{blk} blocks может быть собственный sequence из K_b diffusion steps.

Тогда при одном NFE на step rough total

$$N_{\text{FE}} = \sum_{b=1}^B K_b.$$

Если все $K_b = K$: $N_{\text{FE}} = BK$.

Но actual implementation снова может добавить другие evaluations.

6.11. Один численный пример

Рассмотрим hypothetical system, не result конкретной paper.

Пусть нужно generate $L = 256$ tokens.

Full-sequence diffusion

Пусть sampler имеет 16 refinement steps и один NFE на step.

Тогда $N_{\text{FE}} = 16$.

Каждый NFE обрабатывает все 256 positions: $L_j = 256$.

Получаем $N_{\text{processed}} = 16 \cdot 256 = 4096$.

Block diffusion

Пусть $S = 32$.

Тогда

$$B = \frac{256}{32} = 8.$$

Пусть внутри каждого block четыре denoising steps и один NFE на step.

Тогда $N_{\text{FE}} = 8 \cdot 4 = 32$.

На первый взгляд NFE стало вдвое больше.

Но если каждый evaluation заново прогоняет только 32 current positions, token-processing count current blocks: $N_{\text{processed}} = 32 \cdot 32 = 1024$.

При этом model читает growing cached prefix через attention, так что нельзя сказать, что compute ровно в четыре раза меньше. Но пример показывает главное:

system с большим N_{FE} может пересчитывать меньше token states, а system с меньшим N_{FE} - выполнять тяжёлый full-sequence pass каждый раз.

Поэтому claim «16 steps быстрее 32 steps» без остальных quantities недостаточен.

Пример

Сравним dependency structures.

Full dLLM

State:

the MASK model MASK text

После update:

the diffusion model MASK text

Изменился middle token. Bidirectional attention позволяет всем positions смотреть на него. Old hidden representations потенциально stale. Следующий iteration снова processes sequence.

Block diffusion

Completed block:

the diffusion model

Current block:

MASK MASK text

Current block может смотреть на completed block.

Но completed block не смотрит обратно на current block.

Когда current block меняется:

generates MASK text ,

cached K/V для the diffusion model остаются valid.

Именно one-way dependency boundary создаёт safe reuse.

Где здесь легко ошибиться

Ошибка 1. «Нет KV cache - значит почти нет memory».

Нет. Это утверждение говорит только об одной component persistent state и игнорирует workspace, weights, HBM traffic и repeated computation.

Ошибка 2. «Full dLLM может просто reuse K/V unchanged tokens».

Даже если token value в position i не поменялось, его higher-layer representation могло поменаться из-за changed bidirectional context.

Ошибка 3. «Bidirectional attention запрещает cache принципиально при любых architectures».

Нет. Оно мешает static cache states, которые зависят от изменяющегося future context. Block-causal structure специально создаёт regions, не зависящие от future block.

Ошибка 4. «Block diffusion - просто autoregression».

Нет. Dependencies между blocks autoregressive, но conditional distribution current block реализуется diffusion process.

Ошибка 5. «Four-step model означает четыре network calls по определению».

Нет. Steps и NFE нужно проверять отдельно.

Ошибка 6. «Four network calls означают, что обработано четыре tokens».

Нет. Один network call dLLM может process всю sequence.

Ошибка 7. «Если article уменьшила sampling steps в 64 раз, wall-clock latency обязана уменьшиться в 64 раз».

Нет. Per-step cost, caching, hardware utilization, batch size и sampler overhead тоже имеют значение.

Ошибка 8. «Block size 32 означает 32 sampler steps».

Нет. Block size - число token positions, не temporal discretization.

Что теперь понятно

Главный systems tradeoff можно сформулировать так.

Full-sequence dLLM покупает parallel prediction внутри step ценой expensive repeated full-context processing.

Block diffusion ограничивает область bidirectional refinement и делает completed prefix invariant. Благодаря этому возвращается exact-style reuse между blocks.

Few-step methods уменьшают другую axis cost - sequential denoising depth.

Эти optimizations могут сочетаться, но воздействуют на разные components.

Вопросы для самопроверки

1. Почему difference token ids 100 и 101 не является осмысленным distance?

Ответ. Потому что ids - arbitrary labels. После permutation vocabulary тот же язык получает другие numerical distances. Modeling rule, основанный на этих distances, меняется без изменения data distribution.

2. Как permutation argument проверяет proposed token geometry?

Ответ. Нужно мысленно произвольно перенумеровать categories. Если generative mechanism меняется только из-за renaming ids, он использует accidental encoding structure вместо свойства categorical data.

3. Почему one-hot representation не даёт ordinary score автоматически?

Ответ. Categorical distribution становится finite mixture point masses на vertices. Это не smooth density относительно ambient Lebesgue measure, поэтому ordinary $\nabla_z \log p(z)$ не определён как в continuous diffusion.

4. Что меняется после Gaussian smoothing one-hot vectors?

Ответ. Появляется smooth continuous density и её score. Но state теперь continuous $z \in \mathbb{R}^V$. Чтобы получить token, нужен дополнительный decoder или projection. Это continuous relaxation, а не бесплатное определение discrete score.

5. Как выглядит masked corruption одного token?

Ответ.

$$q_\tau(\tilde{x}_{\tau,t} \mid x_t) = (1 - \rho_\tau) \mathbf{1}[\tilde{x}_{\tau,t} = x_t] + \rho_\tau \mathbf{1}[\tilde{x}_{\tau,t} = \text{MASK}].$$

6. Почему $\tau = 0$ является clean endpoint?

Ответ. Потому что $\rho_0 = 0$, следовательно token остаётся исходным с probability 1.

7. Почему $\tau = 1$ полностью masked?

Ответ. Потому что $\rho_1 = 1$. Каждая position переходит в **MASK** с probability 1.

8. Как distributed число masked positions при independent corruption?

Ответ.

$$N_{\text{mask},\tau} \sim \text{Binomial}(T, \rho_\tau).$$

Это следует из суммы T independent Bernoulli variables.

9. Чему равны expectation и variance $N_{\text{mask},\tau}$?

Ответ. $\mathbb{E}[N_{\text{mask},\tau}] = T\rho_\tau$,

$$\text{Var}(N_{\text{mask},\tau}) = T\rho_\tau(1 - \rho_\tau).$$

10. Почему marginal kernel ещё не полностью определяет absorbing process?

Ответ. Он задаёт distribution state отдельно на каждом τ , но не joint coupling states между разными noise levels. Absorbing coupling можно получить, один раз sampled uniform threshold U_t для каждой position.

11. Что в masked diffusion учит cross-entropy?

Ответ. В population optimum model восстанавливает posterior clean category $p(X_t \mid \tilde{X}_\tau, \tau)$.

Поэтому output естественно интерпретируется как probability denoiser.

12. Как MDLM weight выглядит через ρ_τ ?

Ответ.

$$w_{\text{MDLM}}(\tau) = \frac{\dot{\rho}_\tau}{\rho_\tau}.$$

При linear $\rho_\tau = \tau$ получаем

$$w_{\text{MDLM}}(\tau) = \frac{1}{\tau}.$$

13. Назовите четыре главных отличия MDLM от BERT.

Ответ.

1. Noise level τ samples по generative schedule, поэтому mask rate меняется.

2. Loss levels weighted согласно diffusion objective.
3. Denoiser является частью specified reverse sampler.
4. Generation начинается с highly или completely corrupted state и iterative denoises его.

14. Делает ли dynamic masking в BERT его diffusion model?

Ответ. Нет. Resampling mask pattern не создаёт probability path, diffusion weighting и reverse generative chain.

15. Что заменяет continuous vector field в discrete transport?

Ответ. Transition rates $r_\tau(x \rightarrow y)$ между states.

16. Как читать master equation?

Ответ.

$$\partial_\tau p_\tau(x) = \sum_{y \neq x} [p_\tau(y)r_\tau(y \rightarrow x) - p_\tau(x)r_\tau(x \rightarrow y)].$$

Первый term - incoming mass в x , второй - outgoing mass из x .

17. Почему master equation сохраняет total probability?

Ответ. Каждый transition $x \rightarrow y$ вычитается из probability state x и добавляется к probability state y . После sum по всем states incoming и outgoing double sums совпадают и сокращаются.

18. Что именно показывает LLaDA для нашей темы?

Ответ. Что masked diffusion construction удалось масштабировать до 8B language model trained from scratch. Это evidence feasibility/scale, но не theorem о superiority над autoregressive models.

19. Почему bidirectional prediction не даёт final answer за один pass?

Ответ. Все candidate predictions первого pass conditioned на один и тот же partially masked state. Они не могут использовать newly accepted predictions друг друга. Чтобы обновлённый context повлиял на remaining positions, нужен следующий evaluation.

20. Почему AR KV cache exact?

Ответ. Causal mask гарантирует, что old prefix representations не зависят от future tokens. После append future token cached K/V prefix остаются valid.

21. Почему full bidirectional dLLM обычно не может сделать то же самое?

Ответ. После изменения одной position она может повлиять через attention на representations всех остальных positions. Даже unchanged token values могут получить changed hidden states и, следовательно, changed K/V.

22. Почему отсутствие KV cache не означает low memory cost?

Ответ. Оно уменьшает только одну persistent-state component. Model всё равно хранит weights и current state, использует workspace и HBM bandwidth, а каждый denoising pass может заново process всю sequence.

23. Как factorizes block diffusion?

Ответ.

$$p(X) = \prod_{b=1}^B p(X^{(b)} | X^{(<b)}).$$

Между blocks dependency autoregressive, внутри each conditional используется diffusion.

24. Почему previous blocks можно cache?

Ответ. Их representations по construction не зависят от current/future block. Изменения внутри current diffusion block поэтому не invalidate K/V completed prefix.

25. Что происходит с caching внутри current block?

Ответ. Current block bidirectional и меняется по denoising iterations, поэтому ordinary static exact K/V cache для всего current block снова не возникает автоматически.

26. Что означает $2.5\times$ у Fast-dLLM v2?

Ответ. Это заявленный авторами empirical speedup до $2.5\times$ относительно standard AR decoding в их experimental setup. Это не universal coefficient для block diffusion.

27. Что корректно сказать про IDLM?

Ответ. IDLM distills many-step DLMs в few-step students. Авторы current paper report substantial reductions sampling steps в своих experiments; конкретное число зависит от teacher, student и sampler.

28. Почему sentence «dLLM генерирует за четыре прохода» некорректен?

Ответ. Four-step result относится к specific IDLM-DCD + sampler configuration. Кроме того, sampler step и NFE не являются synonyms, а один NFE может process много token positions.

29. Может ли model с большим N_{FE} оказаться быстрее model с меньшим?

Ответ. Да. Например, если один system на каждом NFE пересчитывает 1024 positions, а другой только небольшой current block с cached prefix. NFE не кодирует cost одного evaluation.

30. Чем block size отличается от sampler steps?

Ответ. Block size - spatial/combinatorial quantity, число token positions. Sampler steps - temporal quantity, число refinement transitions. У одного block может быть много sampler steps.

Что читать дальше

1. [arXiv:2406.07524](#) - Sahoo et al., “Simple and Effective Masked Diffusion Language Models”, **NeurIPS 2024**. Читать после раздела 3. Главное здесь - увидеть полный derivation MDLM objective, SUBS parameterization и то, почему Rao-Blackwellized bound превращается в mixture MLM losses.
2. [arXiv:2407.15595](#) - Gat et al., “Discrete Flow Matching”, **NeurIPS 2024**. Читать после раздела 4. Нужна не столько benchmark часть, сколько construction discrete probability paths и связь learned posterior с sampling dynamics. Это хороший способ увидеть, что Flow Matching - идея про transport distributions, а не обязательно про vectors в $\mathbb{R}^d$.
3. [arXiv:2502.09992](#) - Nie et al., “Large Language Diffusion Models”, **LLaDA, 2025**. Читать как large-scale case study. Важно смотреть отдельно probabilistic formulation, pretraining objective и inference algorithm, а benchmark statements не превращать в общий verdict «diffusion versus AR».
4. [arXiv:2503.09573](#) - Arriola et al., “Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models”, **ICLR 2025**. Читать ради block factorization, attention pattern и KV caching. Это наиболее прямое продолжение systems question этого занятия.
5. [arXiv:2509.26328](#) - Wu et al., “Fast-dLLM v2: Efficient Block-Diffusion LLM”, **arXiv 2025; ICLR 2026**. Читать ради hierarchical caching и различия между model architecture и actual inference system. Speedup numbers здесь имеет смысл читать вместе с baseline и experimental setup.
6. [arXiv:2602.19066](#) - Li et al., “IDLM: Inverse-distilled Diffusion Language Models”, **ICML 2026**. Читать ради few-step direction. Особенно полезно сравнивать не только final number steps, но teacher, sampler, evaluation metrics и exact configuration, для которой этот number получен.

Переход к занятию 7 теперь можно сформулировать довольно точно. Для continuous и discrete generative models у нас появился общий набор questions:

- какое state space;
- какой probability path;
- какой local prediction target;
- какой training objective;
- какой sampler;
- сколько model evaluations;
- сколько state заново обрабатывается на evaluation;
- какая information reusable между steps.

На занятии 7 эти axes уже можно сравнивать между architectures без подмены одной другой.

Что этот конспект добавляет к контракту нотации

1. t по-прежнему обозначает position в token sequence. Continuous noise time обозначается только τ .
2. Random sequence: $X = (X_1, \dots, X_T)$, а конкретная realization: $x = (x_1, \dots, x_T)$.
3. Clean vocabulary: $\mathcal{V}$.

Для masked corruption используется extended vocabulary

$$\bar{\mathcal{V}} = \mathcal{V} \cup \{\text{MASK}\}.$$

4. Corrupted random sequence на noise level τ : $\tilde{X}_\tau$.

Её отдельная realization в position t : $\tilde{x}_{\tau,t}$.

5. Noise orientation сохраняется из занятия 5: $\tau = 0$ clean data, $\tau = 1$ maximal corruption.

Generation идёт в направлении decreasing τ .

6. ρ_τ - masking probability. В notation курса keep probability: $\kappa_\tau = 1 - \rho_\tau$. В notation MDLM paper этой же величине соответствует α_τ .
7. $N_{\text{mask},\tau}$ - random number masked positions.
8. Probability denoiser пишем как $p_\theta(x_t \mid \tilde{X}_\tau, \tau)$ или D_θ там, где нужно подчеркнуть именно denoising interpretation.

Это categorical distribution, не continuous score field.

9. Для generic discrete dynamics $\mathcal{S}$ обозначает state space, а $r_\tau(x \rightarrow y)$

- transition rate.

10. Master equation:

$$\partial_\tau p_\tau(x) = \sum_{y \neq x} [p_\tau(y) r_\tau(y \rightarrow x) - p_\tau(x) r_\tau(x \rightarrow y)].$$

11. Block index обозначается b : $X^{(b)}$

- current block, $X^{(<b)}$
- all previous blocks.

12. L_{blk} - block size, N_{blk} - number of blocks. Ни одна из этих quantities не является NFE или number of sampler steps.

13. N_{FE} по-прежнему означает фактическое число model/function evaluations.

Если нужно учитывать repeated sequence processing, используем L_j для числа заново обработанных token positions на evaluation j и

$$N_{\text{processed}} = \sum_{j=1}^{N_{\text{FE}}} L_j.$$

14. Sampler step и NFE отныне считаются разными quantities по умолчанию. Равенство между ними допустимо писать только после проверки конкретного sampler.

15. Главное новое conceptual agreement:

Euclidean score, categorical probability denoiser и CTMC transition rates - разные mathematical objects. Их нельзя отождествлять только потому, что все они используются для reverse generative dynamics.

И ещё один systems agreement:

Отсутствие KV cache не является само по себе преимуществом по memory или compute. Нужно отдельно считать persistent state, workspace, traffic, NFE и объём recomputation.